Data Creators

Instructor plan · Four days of teaching

Contents

Course timetable 2
- Day 1 Product discovery and effective AI use 2
- Day 2 Mappings service levels and implementation foundations 3
- Day 3 dbt practical and consolidation 4
- Day 4 Bitemporal loading and Airflow orchestration 5

Daily learning objectives and teaching summaries 6
- Day 1 Product discovery and effective AI use 6
- Day 2 Mappings service levels and implementation foundations 9
- Day 3 dbt practical and consolidation 12
- Day 4 Bitemporal loading and Airflow orchestration 14

Before the course 17

Course timetable

Day 1 Product discovery and effective AI use

Time	*Session*	*Instructor actions*
09:00–09:15	Welcome and introductions	Play the leadership welcome; cover logistics and introduce the facilitators.
09:15–09:30	Challenge 1 Introductions and pain points	5 minutes on teams, roles and hobbies; 10 minutes listing products and pain points. Keep written notes.
09:30–09:45	Sponsor contribution	Introduce data excellence, discovery, trust and access. Clarify consumer, owner, platform and creator responsibilities.
09:45–10:05	Table share-out	Collect each table's products and problems. Keep a visible list for later challenges.
10:05–10:15	What makes a data product	Explain promises, assets and alignment. Introduce Friday's capstone.
10:15–10:30	Break	
10:30–10:40	Discovery and product justification	Elicit purpose, users, alternatives, feasibility, value, adoption, cost and priority.
10:40–11:00	Challenge 2 Justify a product	Groups select a product and justify the business need.
11:00–11:25	Debrief	Ask for the hardest question; connect technical difficulties to business consequences.
11:25–11:40	Lifetime value and adoption	Explore ongoing costs, downstream benefits and barriers to adoption.
11:40–11:50	Catalyst demonstration	Show a specific skill, its instructions and its output. Generate a PRD draft.
11:50–12:00	Challenge 3 Compare independent PRDs	Partners describe the same product independently, then compare their generated requirements.
12:00–13:00	Lunch	
13:00–13:40	AI foundations and effective practice	Explain models, tools, harnesses and skills; demonstrate settings, context management and output review.
13:40–13:55	Break	
13:55–14:25	Challenge 4 Interview and generate	In threes, interview producer and consumer; use the interview transcript to generate a PRD, then a contract.
14:25–14:45	Debrief and requirements discovery	Inspect the outputs; compare interviews, observation and surveys.
14:45–15:00	Challenge 5 Explore the lifecycle	Spend 10 minutes exploring phases and prerequisites, then 5 minutes sharing corrections and saving the artefacts.

Day 2 Mappings service levels and implementation foundations

Time	*Session*	*Instructor actions*
09:00–09:20	Mappings and service commitments	Explain mapping rules and the distinction between SLIs, SLOs and SLAs.
09:20–09:45	Challenge 6 Extend the contract	Query Catalyst, then add realistic mappings and commitments to yesterday's contract.
09:45–10:00	Break	
10:00–10:15	Review and present a contract	Review a submitted contract in Markdown and generate a readable HTML version.
10:15–10:45	Mapping detail and platform architecture	Resolve mapping ambiguities; explain tool roles, storage, compute, lakes, lakehouses and Delta.
10:45–11:00	Break	
11:00–11:25	Docker demonstration	Finish the HTML review; show images, containers, Dockerfiles and the development environment.
11:25–12:00	Challenge 7 Docker practical	Run, inspect, build and rerun a small image. Use the short classroom brief below.
12:00–13:00	Lunch	
13:00–13:15	Contract to code and Databricks setup	Brief Challenge 8; show authentication, training schema and output location.
13:15–14:05	Challenge 8 Generate and investigate	Generate dbt and Airflow code from the contract. Explain it and run as far as possible.
14:05–14:15	Break	
14:15–14:30	Debrief	Compare code, execution, design choices and corrections.
14:30–14:50	Review another pair's generated code	Trace one contract rule through the code; identify an assumption and suggest a check.
14:50–15:00	Corrections and closing discussion	Make one correction and share its reason. Record questions for the dbt practical.

Day 3 dbt practical and consolidation

Time	*Session*	*Instructor actions*
09:00–10:00	Challenge 9 dbt practical	Launch the assigned dbt project. Explain the selected tasks and expected checks.
10:00–10:15	Break	
10:15–11:00	Continue dbt practical	Circulate, resolve blockers and collect questions for the recap.
11:00–12:00	Guest session	Connect the guest's agreed agenda to the practical work.
12:00–13:00	Lunch	
13:00–14:15	Complete and extend dbt practical	Complete, check and explain the work. Allow roughly three hours of practice across the day.
14:15–14:30	Break	
14:30–14:45	Consolidation	Review the topics below; arrange tomorrow's guest contributions.
14:45–14:55	Test review in pairs	Exchange one mapping and test; decide whether the test would catch an incorrect result.
14:55–15:00	Closing discussion	Share a missing check and save the dbt work for orchestration.

Day 4 Bitemporal loading and Airflow orchestration

Time	*Session*	*Instructor actions*
09:00–09:45	Bitemporal loading	Work through a history example with the guest; discuss loading strategies and edge cases.
09:45–10:00	Orchestration and capstone orientation	Explain orchestration and connect it to the generated pipeline. Preview Friday's work.
10:00–10:15	Break	
10:15–10:45	Airflow demonstration	Show operators, dependencies, callbacks, the UI and Astro blueprints.
10:45–12:00	Challenge 10 Airflow practical	Launch the assigned Airflow project; inspect real task execution and logs.
12:00–13:00	Lunch	
13:00–13:30	Continue Airflow practical	Resolve blockers and regroup at 13:30.
13:30–14:00	CI/CD and deployment context	Use the agreed Harness, Datadog and Astro demonstrations; keep screenshots ready.
14:00–14:15	Break	
14:15–14:35	Release decision in pairs	Review a failed candidate and decide what can be released and what to tell the consumer.
14:35–14:50	Explain a failure and repair	Write a short note another creator could use to investigate the run.
14:50–15:00	Friday preparation and closing questions	Check the starting files and environment; record remaining problems and who will resolve them.

Daily learning objectives and teaching summaries

Day 1 Product discovery and effective AI use

Learning objectives

By the end of the day, learners should be able to:

- Explain what makes a dataset a trusted, reusable product for a particular consumer.
- Justify a product through its purpose, value, feasibility, cost, and adoption needs.
- Interview producers and consumers, then generate and check a PRD and data contract using Catalyst.
- Use AI settings and context deliberately, evaluate outputs, and identify lifecycle phases and build prerequisites.

Summary

Start with learners' existing products and pain points, then examine business justification, lifetime value, and adoption. Introduce Catalyst and effective AI practice before the interview challenge, where learners turn a conversation into a checked PRD and data contract. Close with the lifecycle and build prerequisites.

Product discovery and justification

Use the opening examples to establish that a product must serve a particular consumer and purpose. Ask what promises make the data usable, who owns those promises, and what enforces them.

For **Challenge 2**, ask each group to choose an opening example or propose a new CG product. Give them these questions:

- What business problem or decision does it support, and who needs it?
- Does a suitable product already exist? What do people do today?
- Can we build it, and what would make the work difficult?
- Is the benefit worth the development and ongoing cost?
- Why would consumers adopt it? Why should it be prioritised now?

Have groups note their most difficult or surprising question. Early finishers can examine a second product.

In the debrief, follow the product beyond an API or intermediate system to the person making a decision. Use examples such as advisor/client master data, global differences in definitions, and fragmented engagement data. Press for a clear consumer and business owner.

Ask: "What changes for the person using this?" A platform name or API describes part of the implementation; ask for the decision it supports. Each group should leave Challenge 2 with a named consumer, purpose and reason to proceed, plus one unresolved question.

When discussing lifetime value, include development, licences, data acquisition, maintenance, SME time, coordination, training, and opportunity cost. Elicit benefits such as

consistency, saved consumer time, trust, new insight, and new business opportunities. Connect adoption barriers to practical responses: demonstrations, documentation, training, sponsorship, reliable commitments, and a better fit with consumers' work.

Catalyst and independent prompting

Show how Catalyst combines reusable prompts and templates with scripts and lifecycle workflows. Inspect a contract template, including identity, schema, service commitments, and quality. Demonstrate a specific skill before introducing broader orchestration.

For **Challenge 3**, partners must use the same product but keep their descriptions private until both PRDs are generated. Ask them to compare scope, consumers, requirements, and assumptions. Draw out how input detail changes the result. Have each pair identify one correction or additional piece of context they would provide. Pairs who finish first should exchange one requirement with another pair and ask whether it is precise enough to check. Bring their disagreement to the debrief.

Check: a tidy document can still contain invented fields, owners or approvals. Use the planted contract errors if the live outputs do not give you a clear example. Keep comparison to one or two differences so the activity finishes before lunch.

Effective AI practice

Teach the afternoon concepts in this order:

1. Intelligence, automation, and the distinction between discriminative and generative systems.
2. Training data, prediction, instruction tuning, preference feedback, and checking generated code.
3. Inference and intermediate reasoning.
4. Tool calls, the harness that executes them, agents, and Catalyst skills.
5. Model choice and the trade-offs between quality, speed, and cost.
6. Reasoning settings, context limits, compaction, and deliberate summaries before starting a fresh conversation.
7. Evaluating performance on the learner's actual task, particularly where internal knowledge or novel requirements matter.

Keep the conceptual introduction brief enough to leave time for the interface demonstration. Use a concrete product task to show how context, settings, and iteration affect the output.

Interview challenge

For **Challenge 4**, form new groups of three:

- The **producer** knows the upstream source and its constraints.
- The **consumer** knows the decision, desired result, and working practices.
- The **creator** interviews both and reconciles their requirements.

Choose a familiar product. Give the creator these starting questions:

1. What decision do you make with the data?

2. How do you make that decision today?
3. Who uses the result directly?
4. How is the source generated?
5. What do the terms mean in your work?
6. How do you recognise a usable result?
7. When do you need the result?

Record and transcribe the conversation. Let groups move into collaborative discussion if the role-play stalls. Have them give the interview transcript to Catalyst, generate a PRD, then generate a data contract. They should inspect and correct both.

Keep the interview within the 30-minute activity: about 5 minutes to choose the product and roles, 15 minutes for questions, and 10 minutes to generate and inspect the documents. If generation is slow, retain the interview notes and review one shared output. Use the learner interview questions without assigning its longer 60-minute workshop.

During the debrief, ask:

- Which identifiers, keys, fields, or definitions were wrong or missing?
- Where did the system substitute generic knowledge for internal source knowledge?
- Did the interview leave scope open or lead to an unsuitable table structure?
- What authoritative context resolved the problem?

Compare interviews with observing consumers' work and surveying larger groups. Emphasise follow-up questions, tacit needs, and the difference between stated preferences and actual practice.

Close by reviewing the PRD's purpose, consumers, value, scope, requirements, and success criteria, then the contract's schema, rules, quality, dependencies, glossary, and change management. Include invented revision history among the errors learners should check.

For **Challenge 5**, use 14:45–14:55 to ask Catalyst for the nine lifecycle phases, build-phase prerequisites, and the skills supporting those prerequisites. Use 14:55–15:00 for each group to share one correction and the evidence behind it. Check the answers against the current organisational guidance. Keep the corrected PRD and contract for Day 2.

Before learners leave: save the product notes, interview record, reviewed PRD, contract and open questions together. The team worksheet provides a place to record evidence, assumptions, mappings and owners. Learners should be able to explain one correction they made and why.

Day 2 Mappings service levels and implementation foundations

Learning objectives

By the end of the day, learners should be able to:

- Specify source-to-target transformations and measurable service commitments in a data contract.
- Explain the roles of the tools and services in the paved path.
- Build and run a Docker image and work inside the development environment.
- Generate, explain, and investigate dbt and Airflow code from a contract.

Summary

Extend Day 1's contract with mappings and service commitments, then improve its presentation and resolve specification ambiguities. Explain the platform architecture, practise Docker, and use the contract to generate and investigate dbt and Airflow implementation.

Mappings and service commitments

Begin with source-to-target mapping (STTM): direct mapping, constants, lookups, aggregation, and derived transformations. Use a small source-and-target example to show joins and column rules. Retrieve the organisation's specific transformation categories through Catalyst.

Distinguish a measurable service-level indicator (SLI), its target (SLO), and the agreement governing service commitments (SLA). Elicit latency, throughput, recovery time, delivery delay and time zone, completeness, accuracy, freshness, support turnaround, and availability. Discuss replication, failover, load balancing, and disaster recovery where they help explain a commitment's cost.

For **Challenge 6**:

1. Ask learners to query Catalyst individually about transformation types and service measures.
2. In pairs or small groups, select a contract from Day 1.
3. Specify source and target tables, transformation rules, lookups, and realistic service commitments.
4. Ask Catalyst to extend the contract, then inspect and revise the result.
5. Share the revised contract for review.

Use a submitted contract to demonstrate Markdown preview and editing. Generate a self-contained HTML document with an overview, contents, explanatory prose, diagrams, and navigation. Check whether the presentation makes the specification easier to review.

Return to mapping detail before moving on: resolve identifiers, keys, edge cases, null meanings, and zero/minimum/maximum values.

Ask: "Could someone else implement this rule without asking you what it means?" For a lookup, check the join key and what happens with zero or several matches. For a promise such as "available by 07:30", ask for the time zone, eligible days, measurement point and

action on failure. Finish Challenge 6 with at least one unambiguous mapping and one measurable commitment.

Platform architecture and Docker

Trace the roles of Git, Catalyst, Docker, dbt, Databricks, S3, Airflow/Astronomer, and Unity Catalog. Show where source data, transformations, compute, scheduling, version control, and deployment fit. Explain the standard path and how to handle a justified exception.

Cover relational structure, transactions and ACID guarantees, distributed storage, Hadoop/MapReduce, Spark, and Delta metadata and logs. Use this progression to explain the training environment.

Keep the distinction clear: Git stores versions of code; Docker provides the execution environment; dbt defines transformations and tests; Databricks executes SQL; Airflow coordinates work; Unity Catalog governs and describes registered data. Use the paved-path diagram rather than drawing the whole stack from memory.

In the Docker demonstration, cover:

- Dependencies, reproducibility, isolation, and shared host resources.
- Dockerfile → image → running container.
- Docker Desktop, port mappings, and resource settings.
- Reopening the repository in a development container and recognising its terminal.
- How the environment supports writing dbt and Airflow files, executing work, and committing code.

For **Challenge 7**, use a short classroom version of the Docker project: run a prepared image, inspect a shell inside it, complete a small Dockerfile, build it and run the result. Pull the base image and prepare the supplied app before the session. The full foundation phase takes 60–90 minutes; leave its additional tasks for further practice.

Ask: “Why did the container stop, and what survives when you run a fresh one?” Check that learners distinguish the image, the running process and its writable files. Keep a working example ready for anyone blocked by engine access. Pair learners who need support; offer extensions to experienced participants.

Generate and investigate implementation

For **Challenge 8**, have learners:

1. Give Catalyst their data contract and request a dbt project.
2. Request simulated data matching the contract if source data is unavailable.
3. Add explanatory comments and obtain instructions for running the code and finding outputs.
4. Generate Airflow orchestration matching the contract’s cadence or trigger.
5. Request explanations of the Airflow code and how to execute it.
6. Run what they can, inspect results, and prepare one discovery and one correction for the debrief.

During Challenge 8, pairs who finish the first implementation should choose one further question: how a missing input is handled, whether a lookup can duplicate rows, or what

happens when the same input is processed twice. Ask them to propose an expected result and a check, then report it during the debrief. Experienced learners can investigate a bitemporal macro or Airflow-to-Datadog logging, keeping their findings short enough to explain to the group.

Debrief in this order: dbt as ELT; SQL `SELECT`, Jinja, and YAML; tables, views, and materialisation; transactional and analytical workloads; `run`, `test`, `build`, `debug`, and `seed`; then Airflow DAGs, parallel tasks, Python, blueprints, Databricks operators, and dbt task groups.

Finish by connecting generated code quality to contract quality. Ask learners to explain a design choice, identify an edge case, and revise the specification or implementation. Encourage annotation or dictated corrections when reviewing longer design documents.

Review another pair's generated code

Use 14:30–14:50 for a review in pairs. Exchange the contract and one small piece of generated dbt or Airflow code. Spend 5 minutes explaining the intended behaviour, 10 minutes reviewing, and 5 minutes discussing the findings. Ask the reviewer to:

- Trace one requirement or mapping into the code.
- Find one assumption about keys, nulls, dates, or scheduling.
- Suggest a concrete check and say what should happen if it fails.

If the code cannot run, review the files and expected behaviour. A learner with a working environment can show the output, but everyone should explain their own rule. At 14:50, give pairs 5 minutes to make or record one correction, then use the final 5 minutes to share the reasons and questions for Day 3.

Before learners leave: retain the revised mapping and commitments, generated project, useful commands and unresolved setup issues. Each learner should be able to locate an output table and explain one transformation. Record access problems separately from problems in the generated code.

Day 3 dbt practical and consolidation

Learning objectives

By the end of the day, learners should be able to:

- Work through the dbt pipeline exercises using the training environment.
- Explain models, dependencies, tests, and the impact of source or schema changes.
- Configure materialisation and connect dbt execution to storage and catalogue conventions.
- Review generated artefacts and identify questions to resolve before the capstone.

Summary

Centre the day on the self-paced dbt challenges, with a guest session before lunch and continued practical work afterwards. Use the closing recap to consolidate models, dependencies, tests, storage, catalogues, and materialisation, and prepare the next day's guest contributions.

Practical facilitation and recap

Use the dbt project for the task sequence and completion criteria. Select the foundation tasks first; advanced work has its own starter and time budget. Ask learners to keep a list of commands, configuration changes, questions, and discoveries. Support different working speeds without requiring everyone to stop at the same exercise.

As you circulate, check three things:

- **Sources and staging:** can the learner state what one row means, identify its key and explain the types and units?
- **Joins and models:** are the lookup keys unique, do dates belong in the join, and what happens to unmatched rows?
- **Tests and output:** can the learner show a value checked independently of the model that produced it?

Ask: "Could this join pass its tests and still double the reported value?" Check the row grain, right-side uniqueness, unmatched rows and an independent total. A successful build alone is insufficient.

For the supplied Portfolio Analytics case, keep the measure precise: holdings-only GBP value divided by 100 million synthetic units, with cash excluded. It is not official fund NAV. Use the project's reference checks for the exact results. The eight foundation questions provide short explanations when a learner is stuck.

Lead the recap in this order:

1. **Models and conventions:** staging and integration models, CG naming conventions, and the SQL behind the transformations.
2. **Dependencies and change:** source documentation and the effects of source or schema changes.
3. **Testing and contracts:** schema and value checks, quality rules, contracts, and catalogue information.

4. **Bitemporal loading:** identify questions for the next morning's presentation. Ask a knowledgeable participant or guest to cover the columns and one or two strategies.
5. **Deployment context:** arrange a short contribution on Harness and CI/CD, including interfaces for learners without access.
6. **Catalyst:** review generation, iteration, and the repository's `data_products` area.
7. **Airflow preview:** introduce YAML-configured blueprints as the next day's topic.
8. **Execution and storage:** connect dbt-generated SQL to Databricks/Spark, S3, Delta, columnar storage, metadata, logs, and transactional guarantees.
9. **Catalogue conventions:** distinguish app, domain, and source catalogues.
10. **Quality and materialisation:** revisit checks and configuration at project and model levels.

Use 14:30–14:45 for the recap. Ask learners to explain examples from their own work; use the topic list to fill gaps rather than giving ten separate mini-lectures.

From 14:45–14:55, pairs exchange one mapping rule and its test. The reviewer proposes a wrong result—a duplicated join, missing dated FX rate, or changed unit—and asks whether the test would catch it. Agree one improvement and write the expected failure. During 14:55–15:00, ask for two or three examples, then save the work and questions for orchestration.

Learners who complete the practical before the break should use a copied example to investigate one of those errors and explain its effect on the output. Ask experienced learners to demonstrate both the failure and repair, with an independent value check. Keep the original working project available for the next day.

Before learners leave: keep the working dbt project, model/test results, an independent output check and the outstanding questions for orchestration. Ask each learner to trace one plain-English mapping through its model and test. Confirm the Airflow practice baseline is ready; completing advanced dbt is not a prerequisite for the next day.

Day 4 Bitemporal loading and Airflow orchestration

Learning objectives

By the end of the day, learners should be able to:

- Distinguish business-time and system-time history and compare bitemporal loading strategies.
- Read an Airflow DAG and explain operators, dependencies, parallel tasks, and callbacks.
- Run and inspect Airflow workflows and explain how Astro blueprints are configured.
- Explain how deployment and monitoring support the product and prepare for the capstone.

Summary

Open with bitemporal loading and a worked history example, then introduce Airflow and practise orchestration through the supplied challenges. Follow with deployment and monitoring demonstrations, and finish by preparing learners for Friday's capstone.

Bitemporal loading

Work through the presentation in this order:

1. Review truncate-and-reload, insert overwrite, and slowly changing dimension type 2 (SCD2). Introduce business-time and system-time history.
2. Show the organisation's bitemporal macro and full, incremental, and balance approaches.
3. Query an associate-history example and trace changes over time. Check the joins used in the example.
4. Distinguish system load time from business-effective dates, including delivery timing and time zones.
5. Demonstrate "as of" and "as was" questions, current-record indicators, source corrections, and audit requirements.
6. Trace the Jinja macro calls and change-detection hashes. Explain which behaviour comes from the organisational extension.
7. Compare full snapshots and incremental transactions, business keys, ending prior versions, and inserting new ones.
8. Discuss trading-identity history, balance use cases, and cases requiring custom strategies.

Check understanding by asking learners which time dimension answers a particular business or audit question.

Ask: "The source corrects Monday's value on Wednesday. What was true on Monday, and what did we believe on Tuesday?" Keep business-effective time separate from when the correction became known. Use one concrete timeline before opening the macro code. The temporal reference supplies further examples; the full clinic is separate practice.

Airflow demonstration and practical

Teach the orchestration concepts in this order:

1. **Orchestration:** coordinate when pipeline components run.
2. **Python and operators:** define work, then dependencies. Show the Cosmos dbt task group, other service operators, and Databricks notebook operators.
3. **Dependencies and parallelism:** explain a directed acyclic graph (DAG). Read `task1 >> task2 >> [task3, task4]` as task 1, then task 2, then tasks 3 and 4 in parallel.
4. **Configuration and callbacks:** explain success, failure, and SLA handling. Show how logging and failure handling support diagnosis when the pipeline fails.
5. **Airflow UI:** inspect task states, graph, code, configuration, and logs.
6. **Astro blueprints:** show reusable Python workflows configured through YAML and explain when a custom blueprint may be useful.

For **Challenge 10**, use the Airflow project and its completion criteria. Prepare authentication, worker paths and the supporting dbt baseline before the timed work. Select the foundation route; leave advanced recovery work for a separate session. Establish the Python foundations before moving to blueprint configuration. Encourage learners to inspect task states and logs as they work, and use the Datadog examples where provided. Collect unresolved deployment and monitoring questions for the afternoon contributor. Learners who finish the practical first should explain what would happen if the input were late, the quality check failed, or the run were repeated. Ask for a log or output check supporting their answer; use a prepared example if their environment is blocked.

For the supplied example, follow `source_ready → dbt_build → quality → publication_evidence`. Ask learners to find the current run, inspect a useful log, and check the warehouse output. In the controlled failure, quality must block the final evidence task; removing the cause should allow a checked repaired run.

Ask: “Which run produced this result, and what proves the data is right?” Keep the scheduled interval separate from the fixture’s business date. A parsed DAG or green graph alone does not prove the warehouse result. Keep credentials and path errors separate from the intended data-quality failure.

Deployment and capstone preparation

Use 13:30–14:00 for the deployment contribution. Agree the agenda in advance: where code goes, how deployment is initiated, where execution is visible, and how failures are investigated. Keep screenshots ready so the same discussion can run without live access.

After the 14:00–14:15 break, run a **release decision exercise** until 14:35. Use the platform review example: a candidate has 500 rows but only 497 distinct IDs, and yesterday’s accepted output is still available. Give pairs 10 minutes to decide what should happen next and draft a two-sentence update to the consumer; discuss their answers for 10 minutes. The failed candidate should remain blocked. Whether yesterday’s output is acceptable depends on the consumer’s freshness requirement. Ask what information is missing and who can supply it.

From 14:35–14:50, ask each pair to write a short note about its Airflow failure and repair: the affected run, useful error, cause, action, output check, and anything still unresolved. Swap notes for 5 minutes of that time: could another creator locate the run and explain

what happened? If execution was blocked, use the prepared failure example and distinguish the expected behaviour from what was observed.

Use 14:50–15:00 to check Friday's instructions, starting files and environment, and answer closing questions. Record each remaining problem and the person responsible for resolving it.

Preview Friday's capstone: a minimum certification phase of roughly two to three hours, followed by open-ended stretch work. Prepare the alternative CG-data challenge if offering it to advanced learners. Make sure learners know where the instructions are and have a working environment before they leave.

Before learners leave: retain the DAG, run and task identities, useful logs, output check and a short explanation of the failure and repair. List any setup issue that could prevent starting Friday's work, with the person responsible for the next action.

Before the course

Run the course over four days, 09:00–15:00, with lunch from 12:00–13:00. Plan activities through to 15:00 each day. Keep the closing reviews in place, and use the suggested follow-up questions when learners finish a practical sooner than others. Adjust discussion and support within the session times while preserving the sequence.

Learners develop one product from business need to requirements, contract, transformation code, and orchestration. Carry their examples and artefacts forward between sessions. For the supplied technical exercises, use Portfolio Analytics for dbt and Airflow and make the change of case explicit.

The timetable here sets the delivery order. Learner links open supporting sections; their existing day labels and longer exercise timings do not change this schedule.

- Ask learners to bring an existing data product and examples of its pain points.
- Check access to the training repository, VS Code, Catalyst skills, Docker development environment, and Databricks training workspace.
- Prepare the selected Docker tasks, dbt project, and Airflow project. Read the exercise files and use their instructions and completion criteria during practical work.
- Have paper and pens available. Set up a shared location for product notes, PRDs, contracts, and questions.
- Check a recording and transcription tool for the interview challenge. Apple Voice Memos or a meeting transcript can support the activity.
- Arrange the opening sponsor contribution, the Day 3 guest session, and Day 4 contributions on bitemporal loading and deployment. Confirm the Day 3 guest's agenda and the deployment demonstration in advance.
- Prepare working examples and screenshots for demonstrations where learner access varies. Pair learners whose setup is incomplete with someone who can run the tools.

For editor or environment questions, open the editor guide, runtime setup, or troubleshooting reference. Keep setup instructions there so this plan stays readable during teaching.